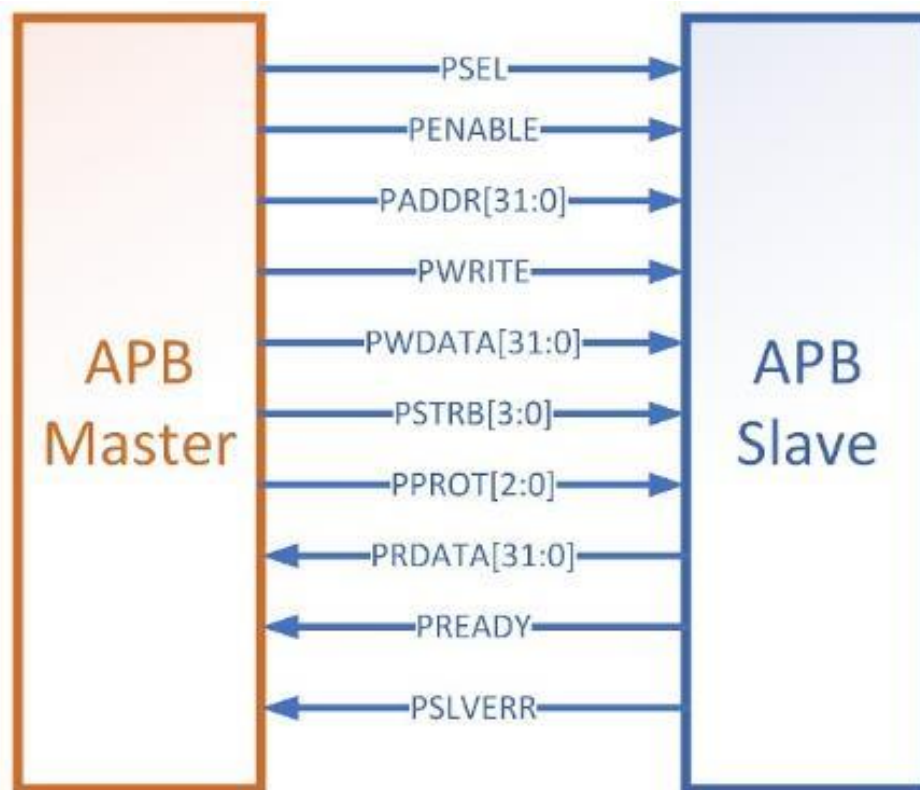


AMBA APB Protocol (APB4)

Design, Implementation & Verification Report



Introduction to AMBA APB Protocol

The AMBA (Advanced Microcontroller Bus Architecture) APB (Advanced Peripheral Bus) protocol is a low-complexity, low-power interface used in modern System-on-Chip (SoC) designs. As part of the AMBA family of protocols, the APB is specifically optimized for communication with peripherals that do not demand the high performance or complexity of buses like AHB (Advanced High-performance Bus) or AXI (Advanced eXtensible Interface).

Importance of the AMBA APB Protocol

The AMBA APB protocol is integral to efficient system design in embedded systems and microcontroller architectures due to its simplicity and power efficiency. By providing a straightforward, non-pipelined communication method, APB allows designers to connect and manage various low-speed peripherals such as timers, UARTs, GPIOs, and more without the overhead associated with more complex bus protocols.

One of the key advantages of the APB protocol is its ability to minimize power consumption, which is critical in battery-operated devices and applications where power efficiency is a priority. The protocol's non-pipelined nature and single-clock-edge operation simplify timing requirements, making it easier to integrate with other system components.

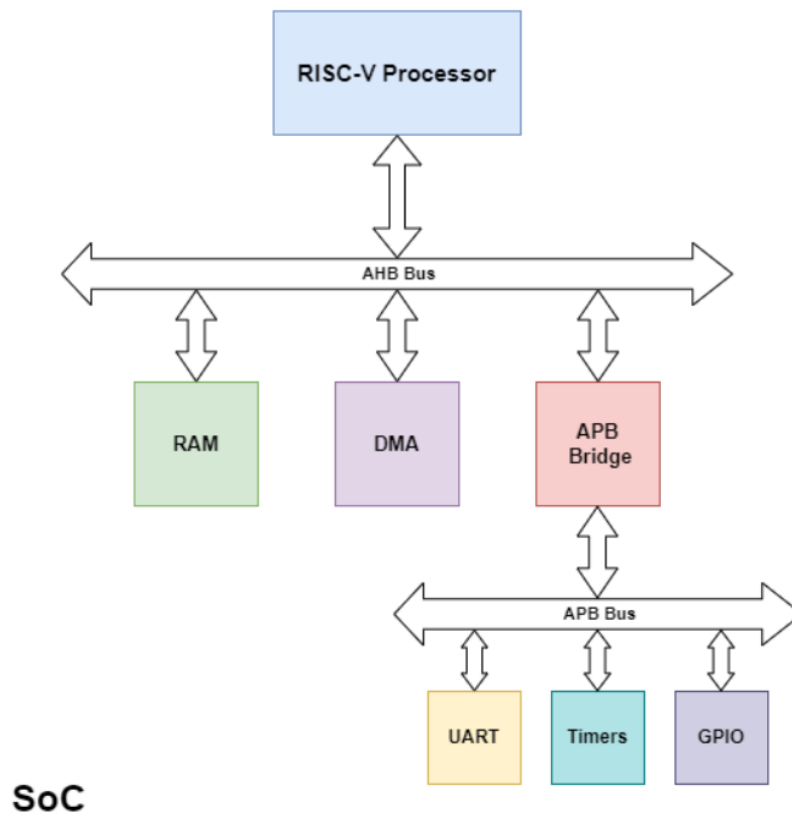
Applications of the AMBA APB Protocol

The AMBA APB protocol is widely used in various applications within SoCs, particularly in areas where simplicity and low power are essential. Common applications include:

- **Peripheral Connectivity:** APB is ideal for connecting low-speed peripherals like GPIO controllers, UARTs, and timers to the main processor. These peripherals typically do not require high-speed data transfers, making APB a perfect fit.
- **Configuration Registers:** Many SoCs use APB for accessing configuration registers that control various aspects of the chip's operation. The low-bandwidth nature of these operations suits the APB's characteristics.
- **Power Management Modules:** APB is often used in power management and clock control modules within SoCs. Its low power consumption aligns with the requirements of these subsystems, ensuring efficient operation.
- **Debug and Control Interfaces:** APB is also used for debug interfaces, control interfaces, and other low-speed data paths within an SoC. Its simplicity makes it easy to implement and integrate into complex designs.

By providing a low-complexity, low-power bus interface, the AMBA APB protocol plays a crucial role in the efficient design and operation of modern embedded systems and SoCs.

SoC Application Using AMBA Protocols



A typical SoC integrating AHB and APB domains

Design Architecture

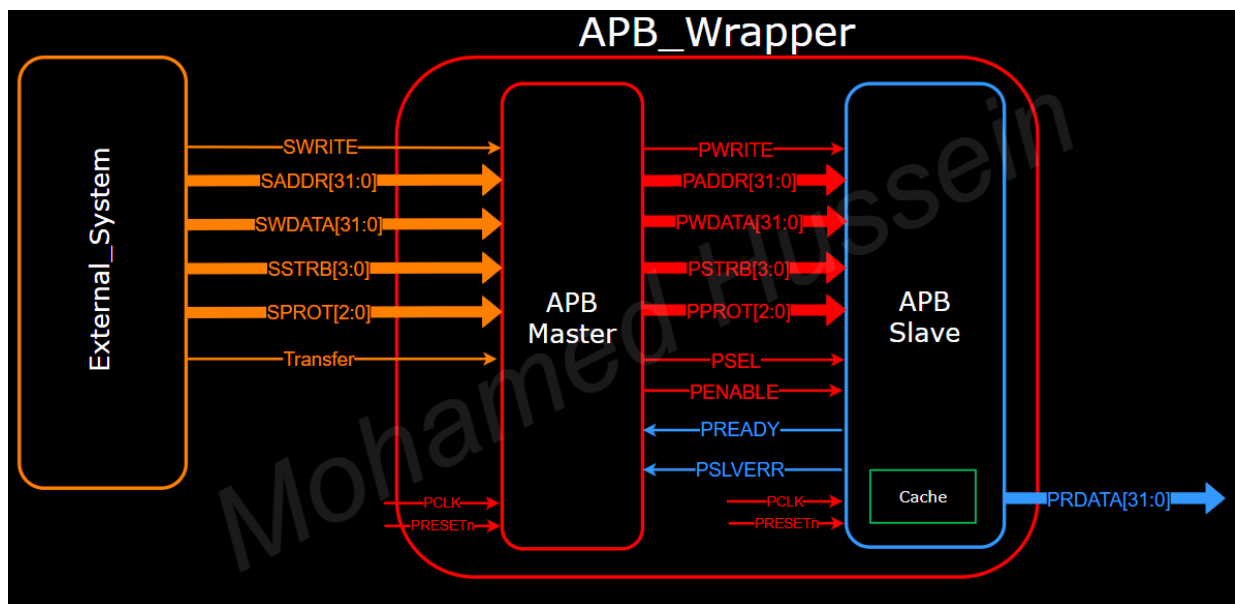


Figure 1 — APB wrapper design architecture

Explanation of Signals

- **PCLK:** The clock signal that times all transfers on the APB. The rising edge of PCLK triggers the transfer of data between the APB bridge and peripheral devices.
- **PRESETn:** A reset signal that is active LOW. It resets the entire APB system, including the peripheral devices connected to the bus.
- **PADDR:** The address bus driven by the APB bridge, which selects the register or memory location within the peripheral that will be accessed during a read or write operation.
- **PPROT:** The protection type signal that indicates the security level of the transaction. It can specify if the transaction is a normal or privileged access, and whether it is secure or non-secure.
- **PSELx:** A select signal generated by the APB bridge to each peripheral. It indicates which slave device is selected for communication and if a data transfer is required.
- **PENABLE:** The enable signal that indicates the second and subsequent cycles of an APB transfer. It differentiates between the setup phase and the access phase of the transfer.
- **PWRITE:** The direction signal that determines if the current operation is a write (HIGH) or read (LOW) operation.
- **PWDATA:** The data bus that carries the write data from the APB bridge to the peripheral. It is only valid when PWRITE is HIGH.
- **PSTRB:** The write strobe signal that indicates which byte lanes are active during a write transfer. Each strobe corresponds to a byte in the write data bus.
- **PREADY:** A signal driven by the peripheral to indicate that it is ready to complete the transfer. It is used to extend the transfer if the peripheral is not ready.
- **PRDATA:** The read data bus driven by the peripheral during a read operation when PWRITE is LOW.
- **PSLVERR:** This signal indicates if there was an error during the transfer. It is not mandatory for peripherals to support this signal.

Note: PSTRB, the write strobe, indicates which byte lanes of the write data bus contain valid data during a write transfer. Each bit in PSTRB corresponds to one byte in the write data bus — for a 32-bit data bus, PSTRB[3] controls bits [31:24], PSTRB[2] controls bits [23:16], and so on.

Write Transfers

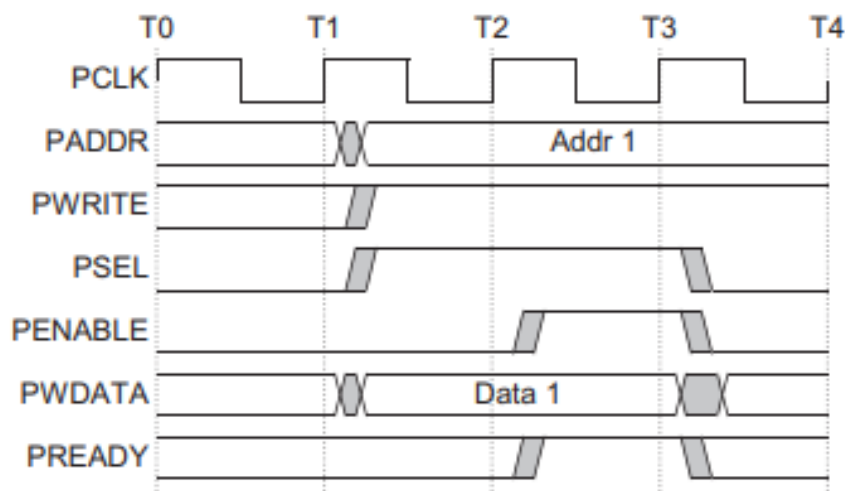


Figure 2 — Write transfer

- **T1 (Setup Phase):** The write transfer begins with the address (PADDR), write data (PWDATA), the write signal (PWRITE), and the select signal (PSEL) being registered at the rising edge of PCLK. This phase is called the Setup phase.
- **T2 (Access Phase):** The enable signal (PENABLE) and the ready signal (PREADY) are registered at the next rising edge of PCLK. When PENABLE is asserted, the Access phase begins, and the slave prepares to complete the transfer.
- **T3 (Completion):** The transfer completes when PREADY is asserted by the slave, indicating that the data has been successfully written. The address, write data, and control signals remain valid until the end of the transfer.

Read Transfers

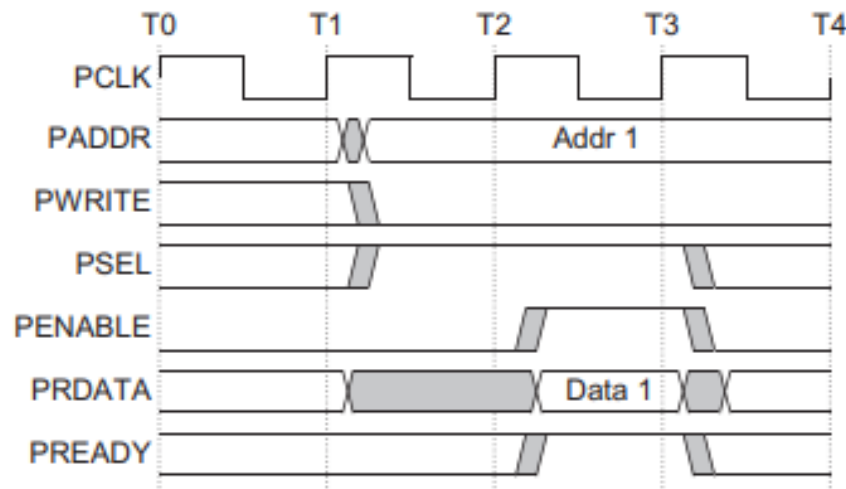
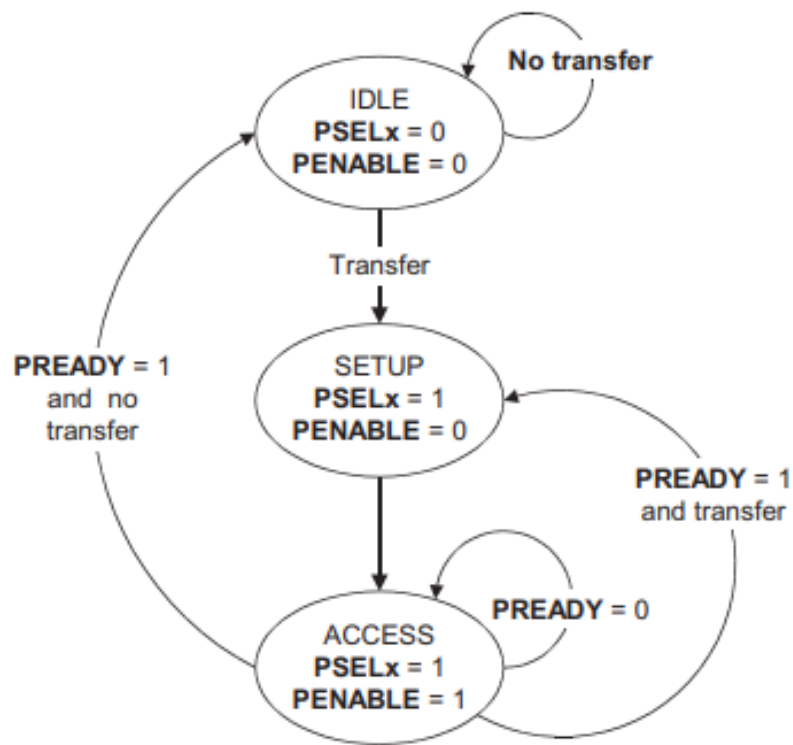


Figure 3 — Read transfer

- **T1 (Setup Phase):** The read transfer begins with the setup of the address (PADDR), write signal (PWRITE = LOW), select signal (PSEL), and enable signal (PENABLE = LOW). This is the Setup phase.
- **T2 (Access Phase):** The enable signal (PENABLE) is asserted, marking the start of the Access phase. The peripheral must provide the read data (PRDATA) before the end of this phase.
- **T3 (Completion):** The transfer completes when the ready signal (PREADY) is asserted by the slave, indicating that the data is available for the master to read.

FSM Explanation



APB protocol finite state machine

The APB protocol FSM has the following states:

1. **IDLE:** This is the default state of the bus. When no transfers are required, the bus remains in the IDLE state.
2. **SETUP:** When a transfer is initiated, the bus moves into the SETUP state where the select signal (PSELx) is asserted. This state only lasts for one clock cycle and prepares the bus for the Access phase.
3. **ACCESS:** In this state, the enable signal (PENABLE) is asserted. The address, write, select, and write data signals remain stable during the transition from the SETUP to the ACCESS state. If the ready signal (PREADY) is asserted by the slave, the bus moves back to the IDLE state if no further transfers are required, or to the SETUP state if another transfer is needed. If PREADY is held LOW by the slave, the bus remains in the ACCESS state until the transfer can be completed.

Verifying Functionality

In the testbench, the external system role is assumed, sending the address, write data, and the transfer signal to the APB master, which then issues the corresponding commands to the APB slave.

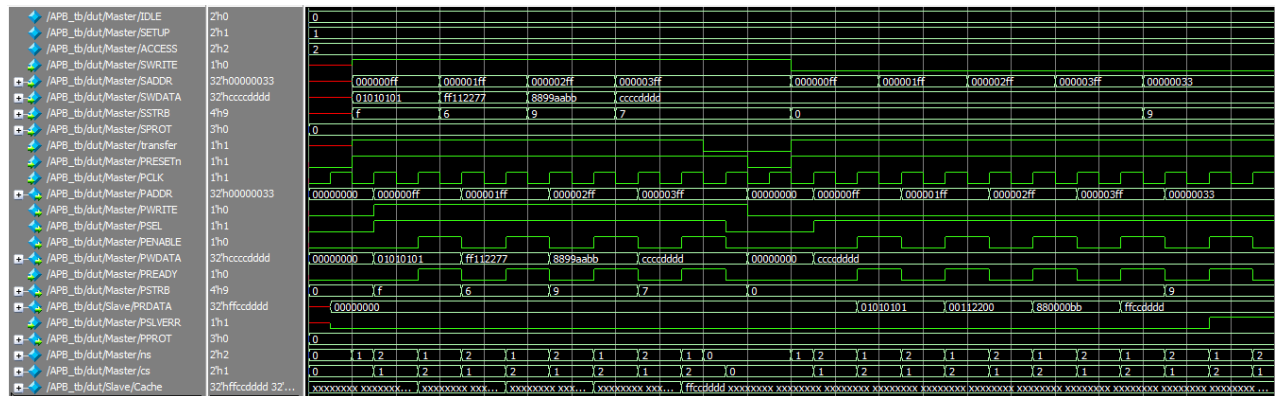
Scenario

- **Writing:** random data is written to several addresses, with each address receiving different data and a different write-strobe signal to verify its functionality.
- **Reading:** after the writing process, the same addresses are read back, followed by a test to verify the slave-error signal functionality.

The following behavior is expected:

- PSEL and PENABLE signals are low in the IDLE state.
- PSEL is high and PENABLE is low in the SETUP state, and the SETUP state is only entered for one clock cycle.
- PSEL and PENABLE signals are high in the ACCESS state.
- The PREADY signal, when high, lasts for one clock cycle and then turns back low.
- The data values in each writing process are written to cache memory at the specified address, formatted according to the PSTRB signal.
- The values written in memory show up in the PRDATA signal.
- Finally, the SLVERR signal is high at the end, due to the test scenario.

Whole Waveform



The screenshot displays the Logic Analyzer interface with four memory dump windows and a timing diagram.

Memory Dump Windows:

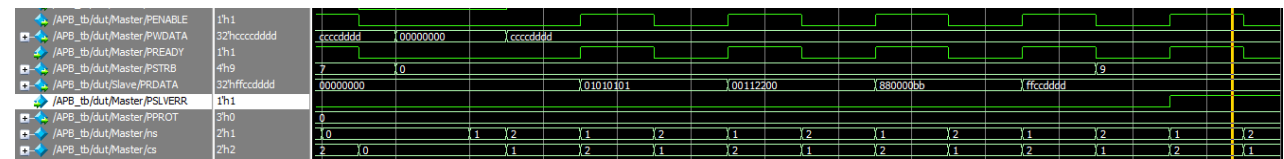
- Window 1:** Addresses 257, 256, 255, 254, 253, 252. Address 255 is highlighted with value 32'h01010101.
- Window 2:** Addresses 513, 512, 511, 510, 509, 508. Address 511 is highlighted with value 32'h00112200.
- Window 3:** Addresses 769, 768, 767, 766, 765, 764. Address 767 is highlighted with value 32'h880000bb.
- Window 4:** Signals APB_tbit dut/Master/ns (2'h), APB_tbit dut/Master/cs (2'h), and APB_tbit dut/Slave/Cache (32'hffccddd). Address 1023 is highlighted with value 32'hffccddd.

Timing Diagram:

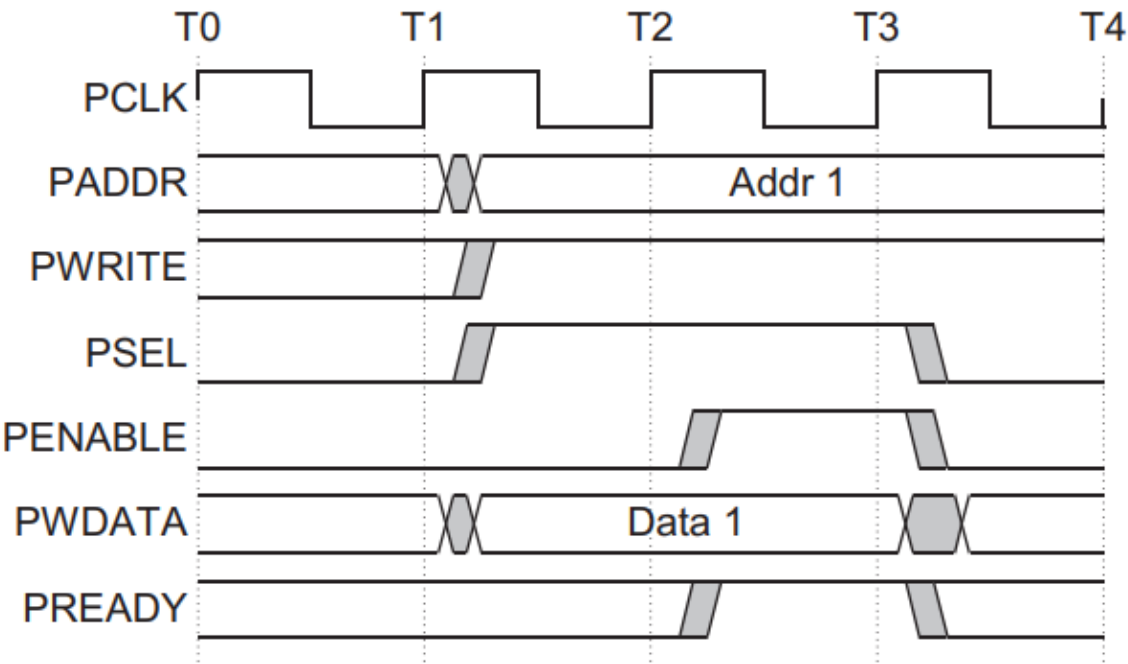
- Signals: APB_tbit dut/Master/PSTRB (1'h0 4'h9), APB_tbit dut/Master/PRDATA (32'hffccddd), APB_tbit dut/Master/PSLVERR (1'h1 3'h0), and APB_tbit dut/Master/PPROT (0).
- The PRDATA signal trace shows a sequence of values: 00000000, 01010101, 00112200, 880000bb, and ffcddd. A red arrow points to the '01010101' value.

Note: the values written in memory differ from the input values due to the write strobe signal, and the last element stored has sign extension.

Values Written in Memory, Shown in PRDATA



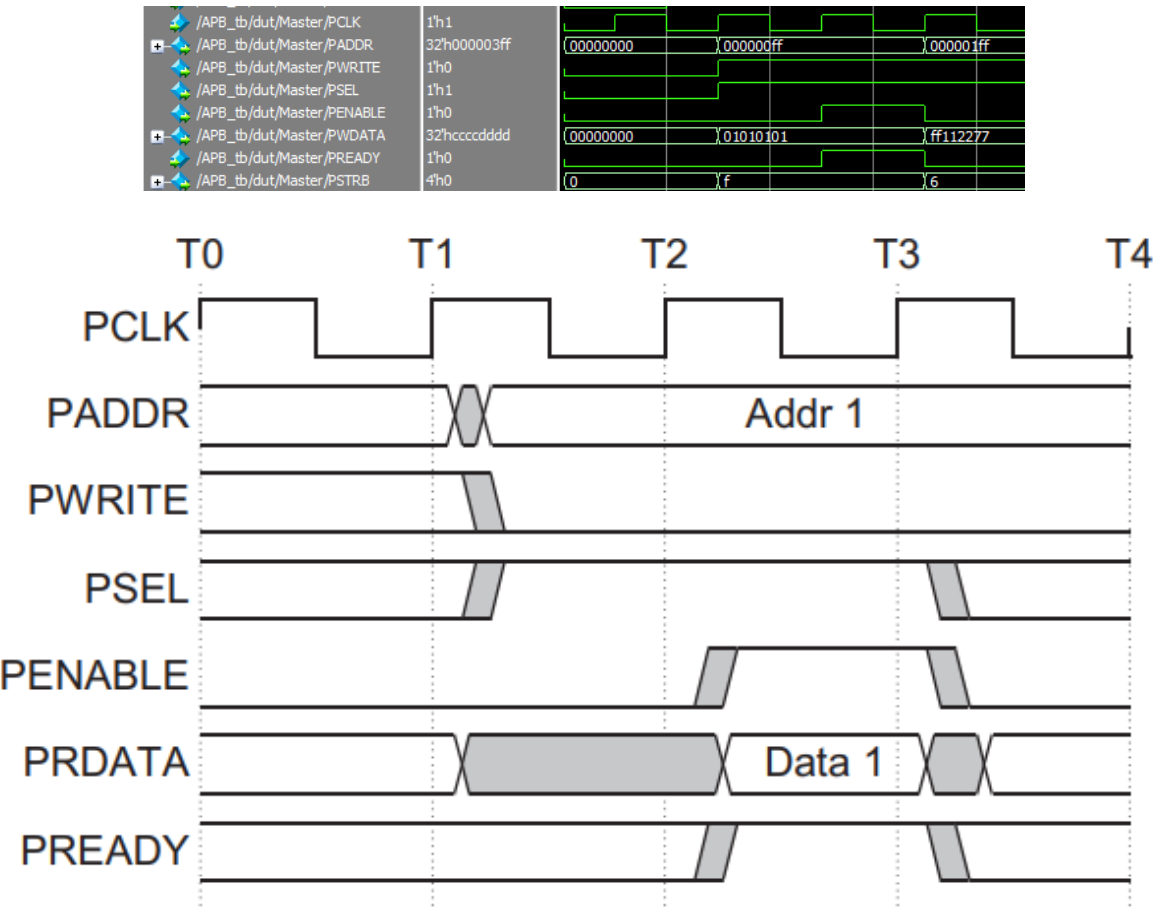
PSLVERR High (PSTRB Has a Value While Reading)



Comparison

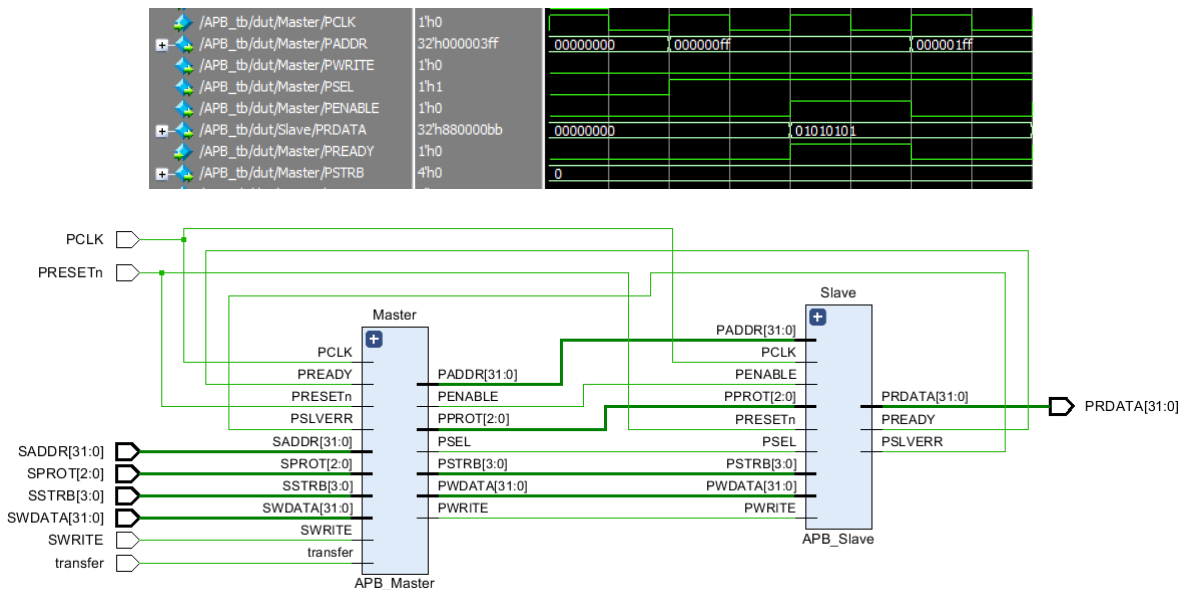
Writing Process

Comparing the APB documentation waveform with the design waveform in the writing process:



Reading Process

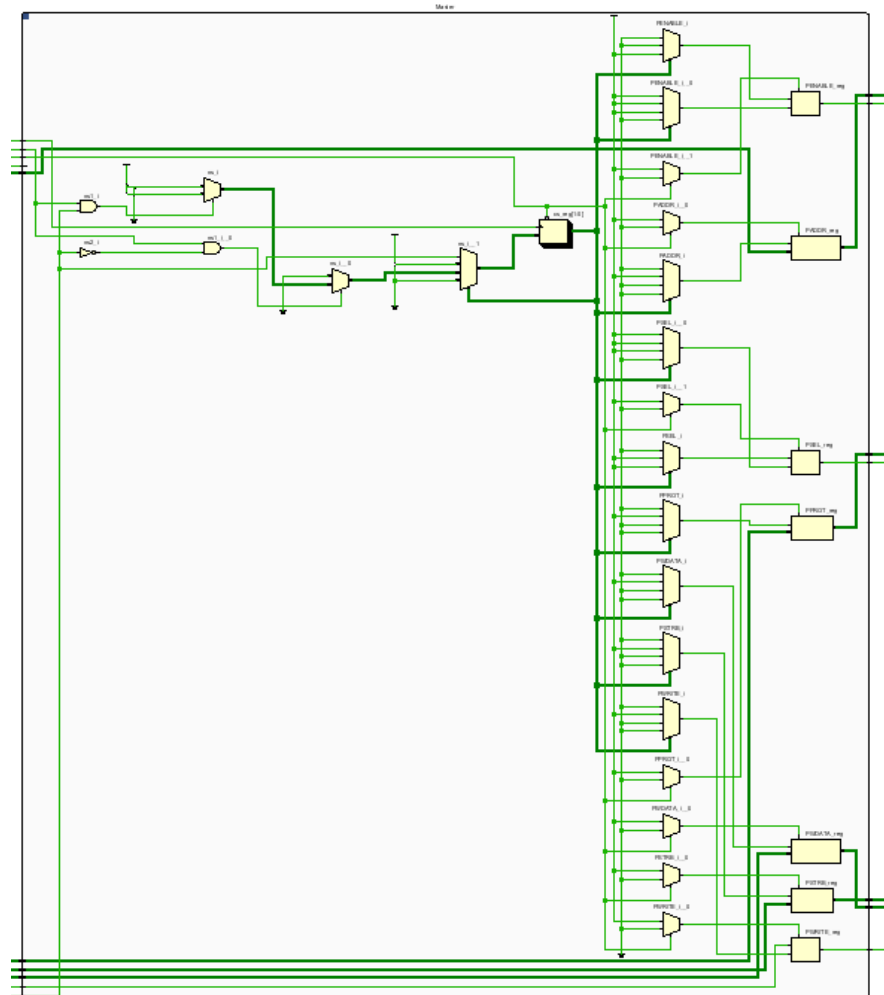
Comparing the APB documentation waveform with the design waveform in the reading process:



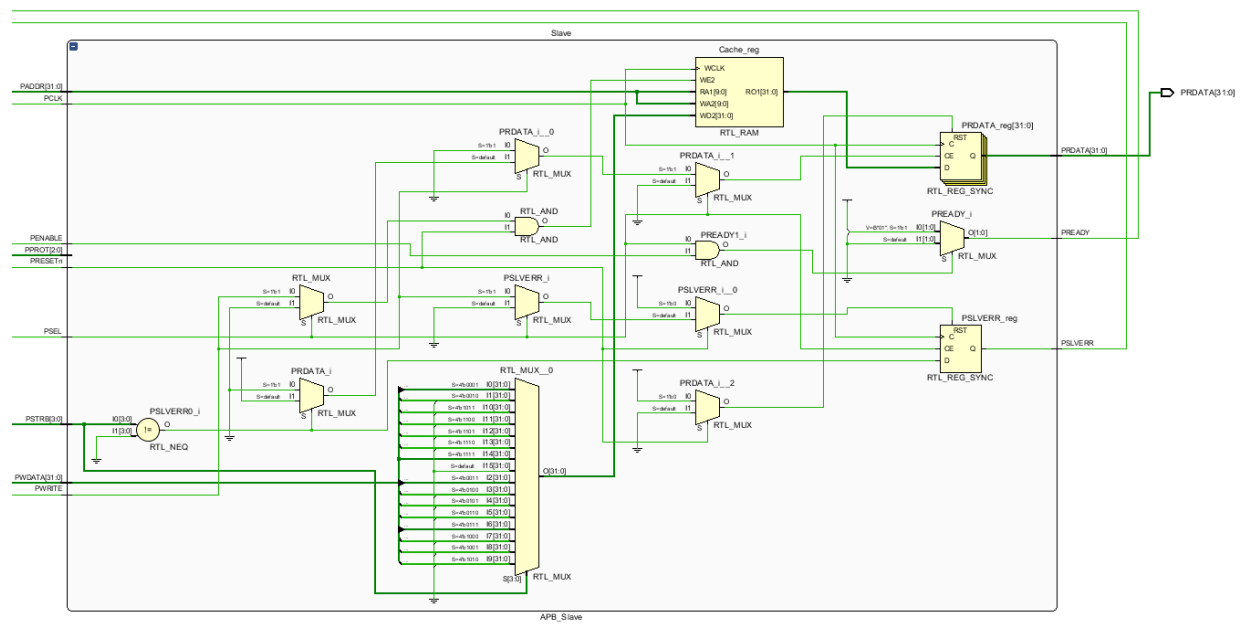
Vivado Implementation

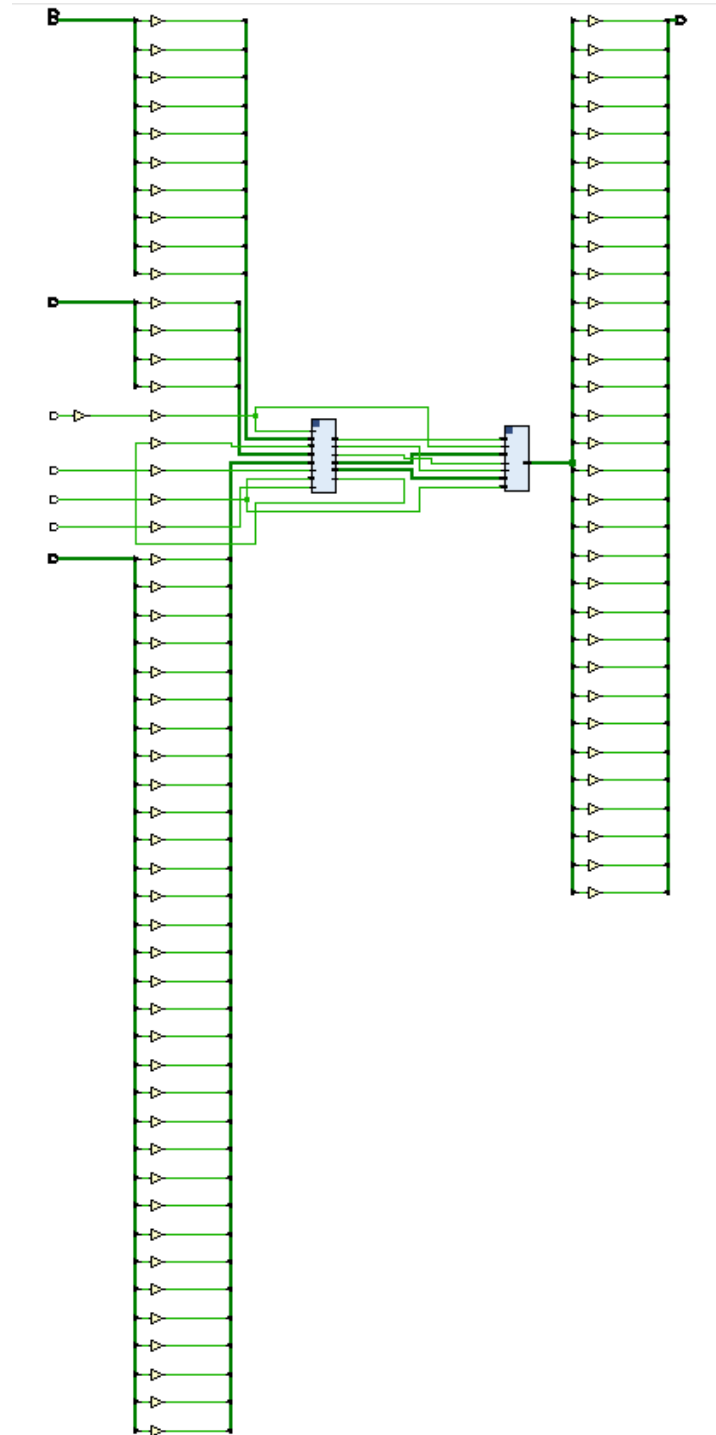
Elaboration

System



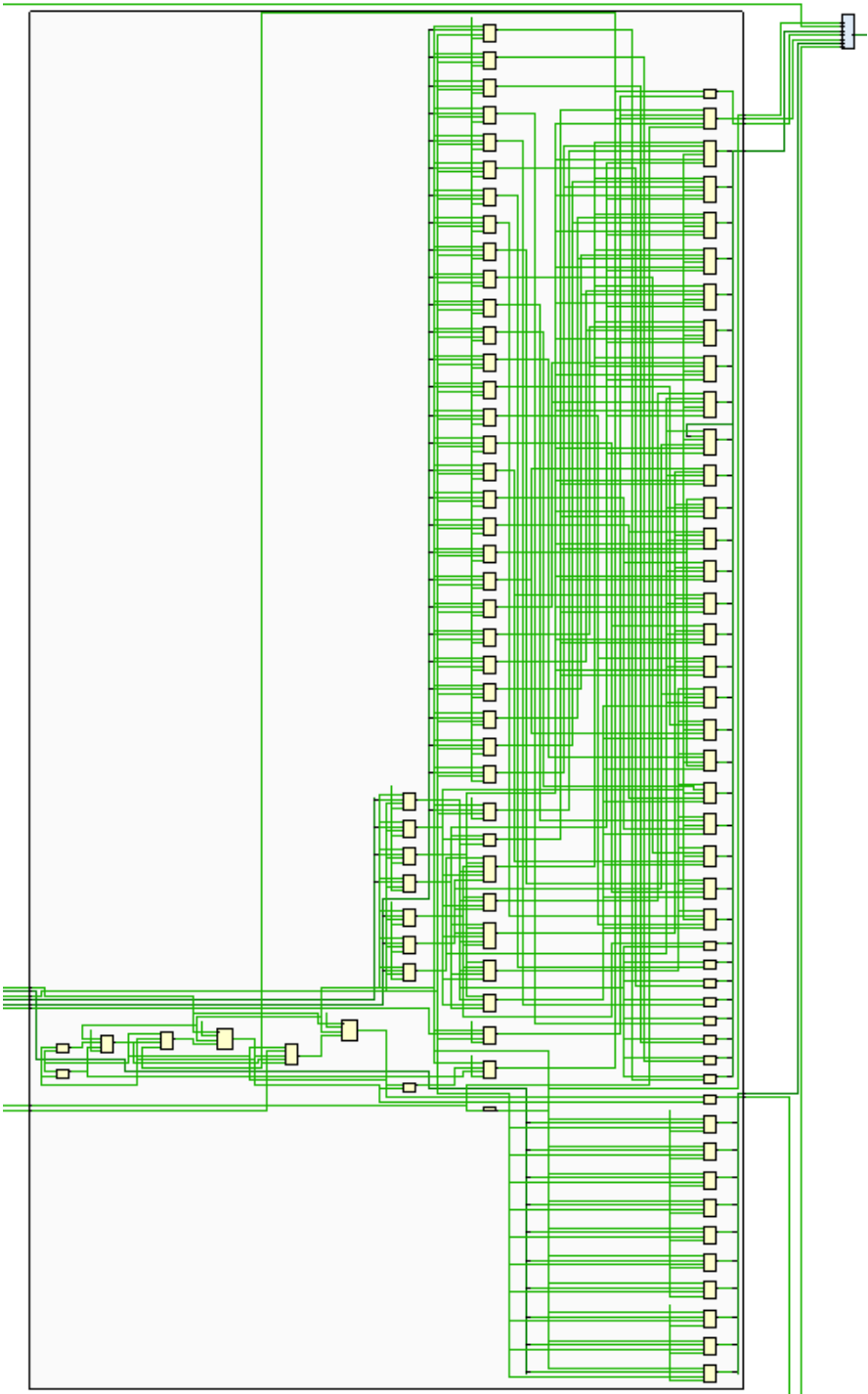
Master



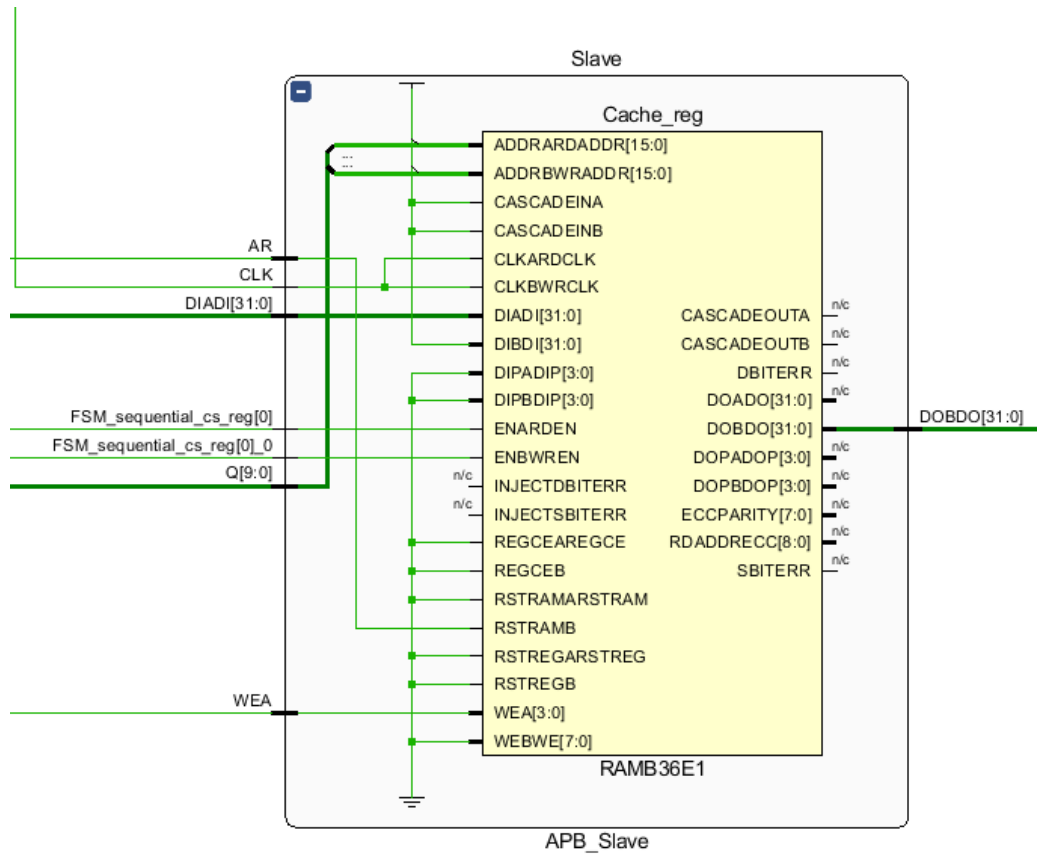
Slave

Synthesis

Sequential — System



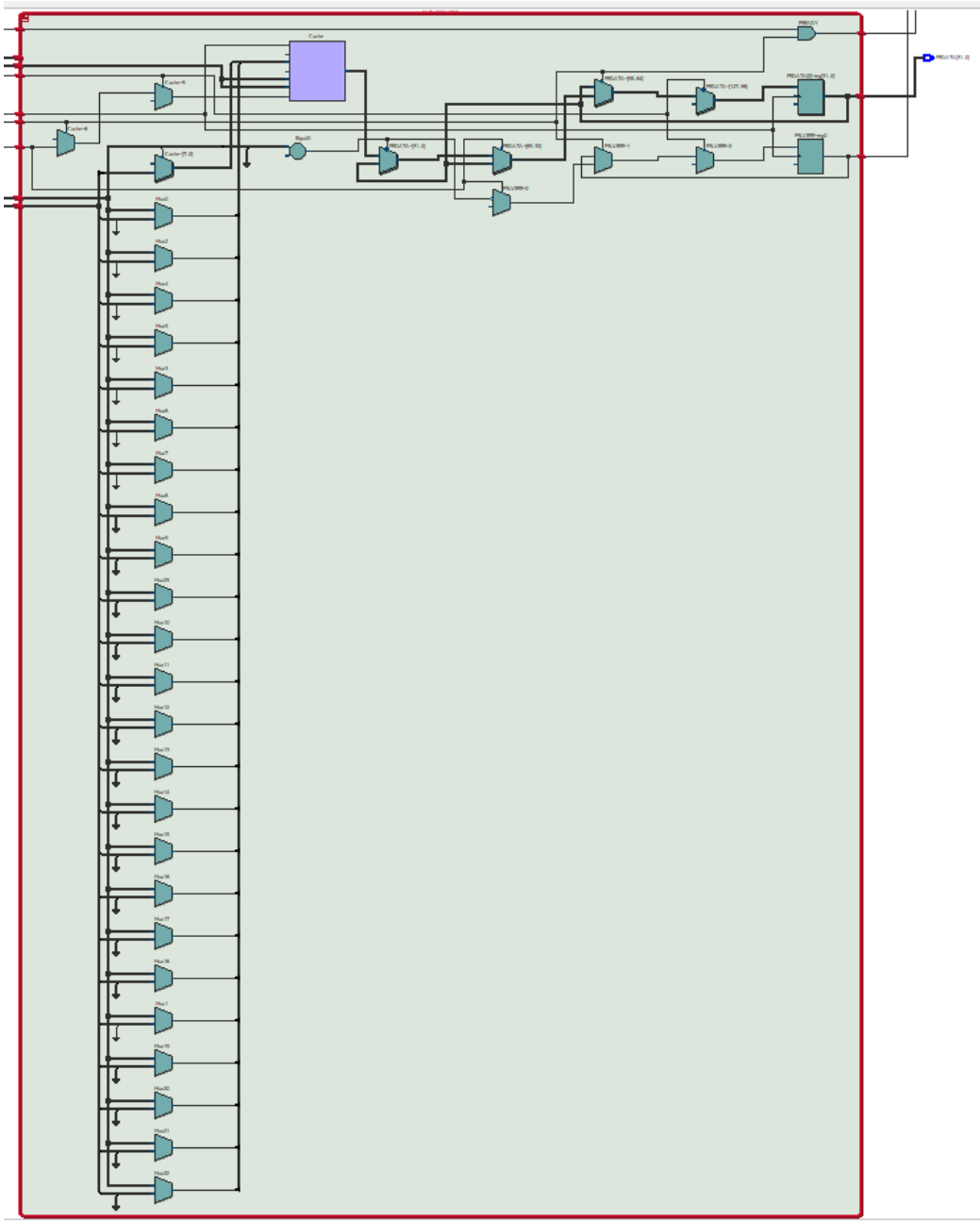
Master



Slave

General Information Timer Settings Design Timing Summary Clock Summary (1) > Check Timing (277) > Intra-Clock Paths Inter-Clock Paths Other Path Groups Timing Summary - timing_seq			
	Setup	Hold	Pulse Width
	Worst Negative Slack (WNS): 8.260 ns	Worst Hold Slack (WHS): 0.297 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
	Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
	Total Number of Endpoints: 2	Total Number of Endpoints: 2	Total Number of Endpoints: 5
All user specified timing constraints are met.			

Timing Summary on a 10 ns Clock Period



Quartus Implementation

Schematic

<<Filter>>								
	Clock Name	Type	Period	Frequency	Rise	Fall	Duty Cycle	Divide by
1	PCLK	Base	10.000	100.0 MHz	0.000	5.000		

Timing (10 ns Period)

Clock

	Fmax	Restricted Fmax	Clock Name	Note
1	108.92 MHz	108.92 MHz	PCLK	

Fmax

<<Filter>>			
	Clock	Slack	End Point TNS
1	PCLK	0.326	0.000

Hold

	Clock	Slack	End Point TNS
1	PCLK	0.819	0.000

Setup

<<Filter>>			
	Clock	Slack	End Point TNS
1	PCLK	4.578	0.000

References

- AMBA APB Protocol Specification.